

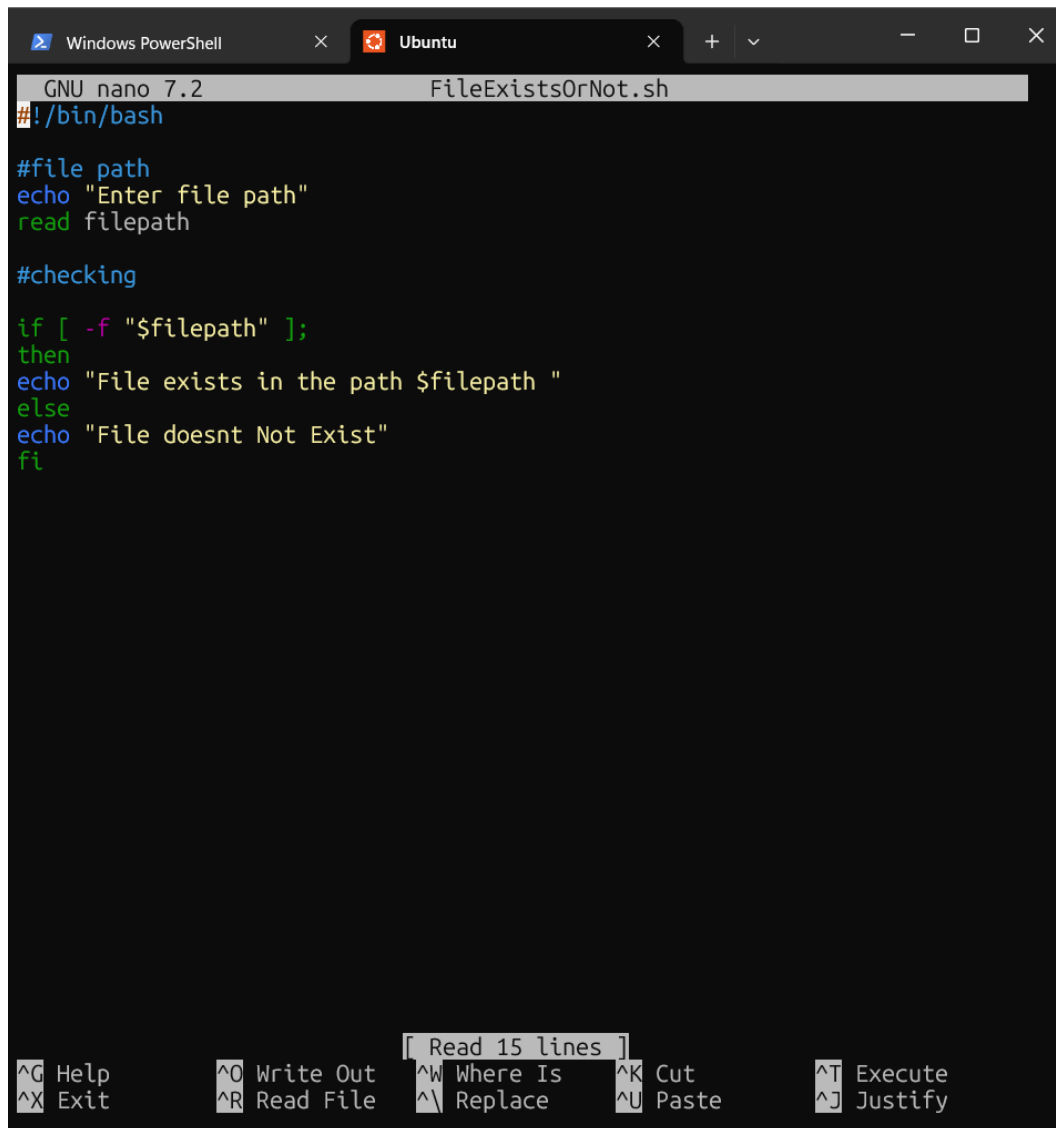
Veeva Integrated Program Assessment – 3

Subject: Linux Operating System concepts

Practical Assessment:

1. Linux scripts to search files:
 - a. Write a script to check the file exists:
 1. Write a script to check if a file exists in the system or not.

Filename: FileExistsOrNot.sh



```
GNU nano 7.2 FileExistsOrNot.sh
#!/bin/bash

#file path
echo "Enter file path"
read filepath

#checking
if [ -f "$filepath" ];
then
echo "File exists in the path $filepath "
else
echo "File doesnt Not Exist"
fi
```

Terminal window showing the nano editor editing FileExistsOrNot.sh. The script checks if a file exists at a given path and prints a message accordingly. The terminal window has tabs for Windows PowerShell and Ubuntu. The Ubuntu tab is active, showing the nano editor. The script content is as follows:

```
GNU nano 7.2 FileExistsOrNot.sh
#!/bin/bash

#file path
echo "Enter file path"
read filepath

#checking
if [ -f "$filepath" ];
then
echo "File exists in the path $filepath "
else
echo "File doesnt Not Exist"
fi
```

At the bottom of the terminal window, there is a status bar with various keyboard shortcuts: ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify. A tooltip "Read 15 lines" is visible over the status bar.

Output:

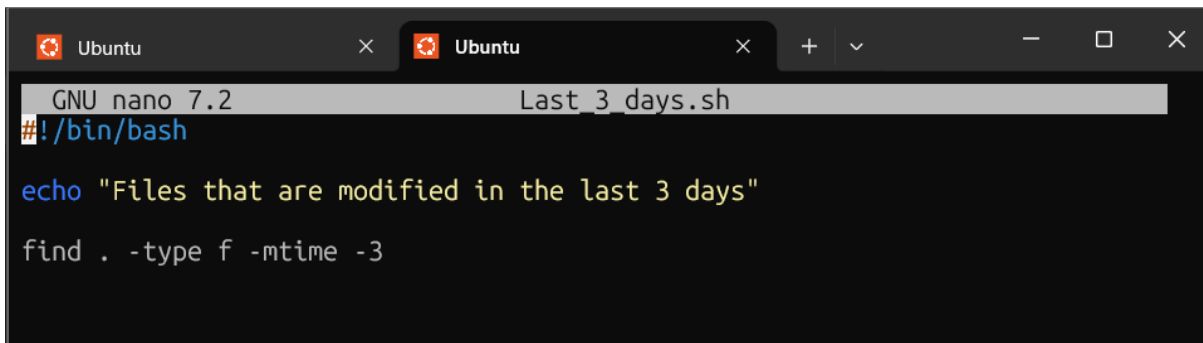
```
harika@HarikaKadiyam:~$ cd /mnt/c/users/harik/oneDrive/Desktop
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop$ chmod +x "FileExistsOrNot.sh"
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop$ ./FileExistsOrNot.sh
Enter file path
/mnt/c/users/harik/oneDrive/Desktop/rtcamp.pdf
File exists in the path /mnt/c/users/harik/oneDrive/Desktop/rtcamp.pdf
```

```

harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop$ ./FileExistsOrNot.sh
Enter file path
/mnt/c/Automation/SeleniumWebDriver/SeleniumWebDriver/src/test/java/Day1_FirstTestCasesSample/First_TestCase.java
File exists in the path /mnt/c/Automation/SeleniumWebDriver/SeleniumWebDriver/src/test/java/Day1_FirstTestCasesSample/First_TestCase.java

```

2. Check whether if the file exists in any part of the system or not.
- b. Write a Linux script that searches for files modified within the last 3 days within the current directory.
Filename: Last_3_days.sh



```

GNU nano 7.2 Last_3_days.sh
#!/bin/bash

echo "Files that are modified in the last 3 days"

find . -type f -mtime -3

```

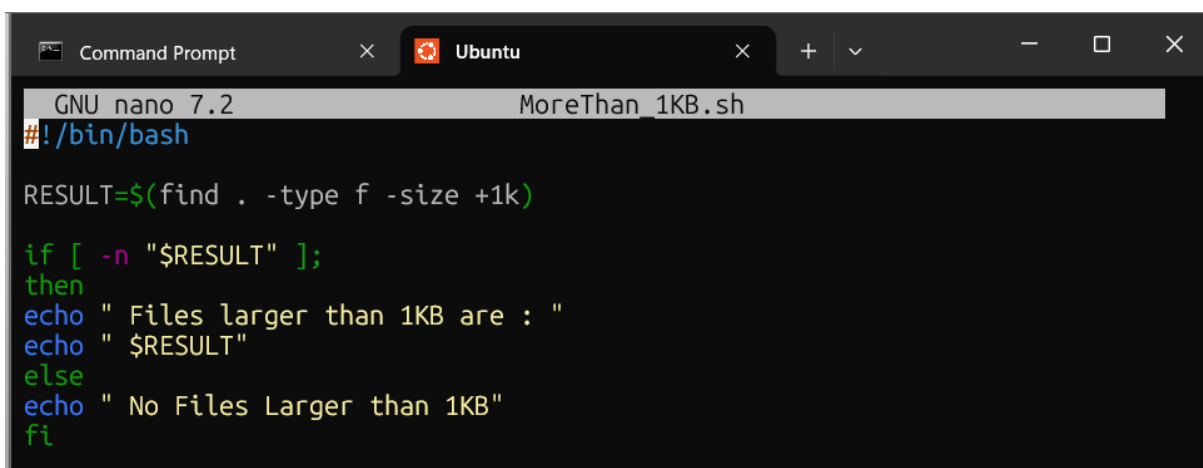
Output:

```

harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ chmod +x Last_3_days.sh
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ ./Last_3_days.sh
Files that are modified in the last 3 days
./FileAnywhere.sh
./FileExistsOrNot.sh
./Last_3_days.sh

```

- c. Write a Linux script that searches for files larger than 1KB in the current directory.



```

GNU nano 7.2 MoreThan_1KB.sh
#!/bin/bash

RESULT=$(find . -type f -size +1k)

if [ -n "$RESULT" ];
then
echo " Files larger than 1KB are : "
echo " $RESULT"
else
echo " No Files Larger than 1KB"
fi

```

Output:

```
Command Prompt x Ubuntu x + v - □ x
harika@HarikaKadiyam:~$ cd /mnt/c/users/harik/oneDrive/Desktop/Linux_Test
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ ./MoreThan_1KB.sh
No Files Larger than 1KB
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$
```

- d. Write a Linux script that searches for files with a specific extension in the current directory.

Filename: SearchByEx.sh

```
Command Prompt x Ubuntu x + v - □ x
GNU nano 7.2 SearchByEx.sh
#!/bin/bash

# either extension given or not
if [ -z "$1" ]; then
    echo "Usage: $0 <extension>"
    exit 1
fi

extension=$1

echo "Files with extension: .$extension"

found=false

for file in *.$extension; do
    if [ -e "$file" ]; then
        echo "$file"
        found=true
    fi
done

if [ "$found" = false ]; then
    echo "No files with .$extension"
fi
```

Output:

```
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ ./SearchByEx.sh
Files with extension: .sh
FileAnywhere.sh
FileExistsOrNot.sh
Last_3_days.sh
MoreThan_1KB.sh
SearchByEx.sh
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$
```

2. Text processing a searching for patterns:

- a. Write a Linux script that searches for a specific string in a given text file.

Filename: SearchString.sh

```
Command Prompt x Ubuntu x + v - □ x
GNU nano 7.2 SearchString.sh
#!/bin/bash

# Check if correct number of arguments are provided
if [ $# -ne 2 ]; then
    echo "Usage: $0 <filename> <search_string>"
    exit 1
fi

filename=$1
search_string=$2

# Check if file exists
if [ ! -f "$filename" ]; then
    echo "File '$filename' does not exist."
    exit 2
fi

# Search for string in the file
echo "Searching for '$search_string' in $filename..."
grep -n "$search_string" "$filename"

# Check if string was found
if [ $? -ne 0 ]; then
    echo "String not found."
fi
```

Output:

```
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ ./SearchString.sh textfile.txt "customization"
Searching for 'customization' in textfile.txt...
3:This article investigates the effects of product customization on user experience and spending behavior in online shopping environments. It emphasizes that platforms offering enhanced customization features improve user satisfaction and increase engagement, showing that customization is a key factor in boosting online sales and loyalty.
```

- b. Consider HTML file as input and write a Linux script to extract text between specified HTML tags.
Filename: ExtractText.sh

```
Command Prompt  Ubuntu
GNU nano 7.2 ExtractText.sh
#!/bin/bash

# Check if correct number of arguments are provided
if [ $# -ne 2 ]; then
    echo "Usage: $0 <html_file> <tag_name>"
    exit 1
fi

html_file=$1
tag=$2

# Check if file exists
if [ ! -f "$html_file" ]; then
    echo "File '$html_file' does not exist."
    exit 2
fi

# Extract text between the specified HTML tags
echo "Extracting text between <$tag> and </$tag>..."
grep -oP "(?<=<$tag>).*?(?=</$tag>)" "$html_file"
```

Output:

```
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ ./ExtractText.sh sample.html title
Extracting text between <title> and </title>...
My Website
```

3. Write a script that logs a message to a file log.txt:
 - a. Write a script that logs errors to a separate file error.txt.
Filename: kogError.sh

```
Command Prompt  Ubuntu
GNU nano 7.2 logError.sh
#!/bin/bash

# Error log file
error_log="error.txt"

# Clear previous error log
> "$error_log"

# Example commands
echo "Starting script..."

# Command 1 (successful)
ls /tmp

# Command 2 (intentional error for demonstration)
ls /nonexistent_directory 2>> "$error_log"

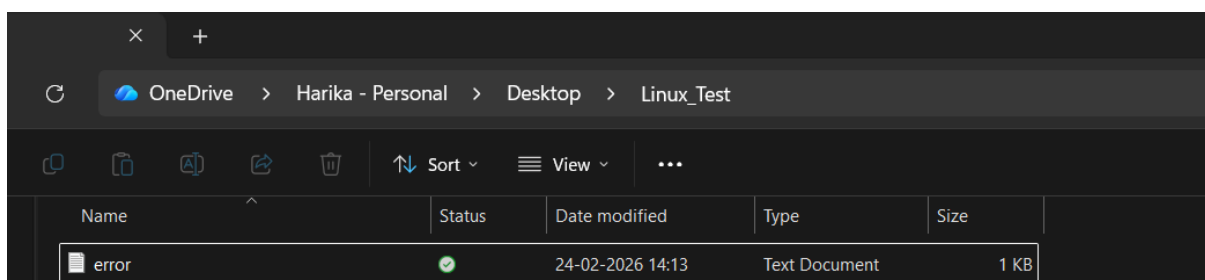
# Command 3 (another example)
cat /etc/passwd

# Command 4 (another intentional error)
grep "something" /nonexistent_file 2>> "$error_log"

echo "Script completed. Check '$error_log' for errors."
```

Output:

```
my website
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ nano SearchString.sh
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ nano ExtractText.sh
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ touch logError.sh
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ nano logError.sh
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ chmod +x logError.sh
harika@HarikaKadiyam:/mnt/c/users/harik/oneDrive/Desktop/Linux_Test$ ./logError.sh
Starting script...
snap-private-tmp
systemd-private-80dfd30faccd41a0b2fa457addf7d08f-redis-server.service-RiuuSo
systemd-private-80dfd30faccd41a0b2fa457addf7d08f-systemd-logind.service-WmTrkb
systemd-private-80dfd30faccd41a0b2fa457addf7d08f-systemd-resolved.service-SNz5nr
systemd-private-80dfd30faccd41a0b2fa457addf7d08f-systemd-timesyncd.service-uj7LFW
systemd-private-80dfd30faccd41a0b2fa457addf7d08f-wsl-pro.service-RYYA2U
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:996:996:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
syslog:x:102:102::/nonexistent:/usr/sbin/nologin
```



The screenshot shows a web browser window with the address bar displaying 'OneDrive > Harika - Personal > Desktop > Linux_Test'. Below the address bar is a toolbar with icons for copy, paste, delete, and other file management actions. A table lists the files in the folder:

Name	Status	Date modified	Type	Size
error		24-02-2026 14:13	Text Document	1 KB

Subject: SDLC, Agile Software Model & Scrum.

Viva Questions:

1. What is the Agile Manifesto, how we control the product development?

The **Agile Manifesto** is a brief document created in 2001 that outlines four core values and 12 principles for software development. It prioritizes:

- **Individuals and interactions** over processes and tools.
- **Working software** over comprehensive documentation.
- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a plan.

Control in Agile is not achieved through rigid schedules, but through **Empirical Process Control**, which relies on three pillars:

- **Transparency:** All aspects of the process must be visible to those responsible for the outcome (e.g., Task Boards, Burndown charts).
- **Inspection:** Regular checks on progress and artifacts to detect undesirable variances (e.g., Daily Stand-ups, Sprint Reviews).
- **Adaptation:** Adjusting the process or the product immediately if a deviation is found (e.g., Sprint Retrospectives, Backlog Refinement).

2. Explain and analyze the differences between burn-down and burn-up charts.

Both charts track progress, but they provide different perspectives:

Feature	Burn-down Chart	Burn-up Chart
Primary Focus	Shows how much work is remaining .	Shows how much work is completed .
Lines	One line (Actual work remaining) vs. Ideal line.	Two lines: Total Scope (Goal) and Completed Work.
Scope Creep	Hidden. If scope is added, the line simply goes up or stays flat, making it look like the team is slow.	Visible. You can see the "Total Scope" line move up, showing exactly why the finish line moved.
Best Use Case	Daily tracking within a single Sprint to see if the team will finish on time.	Long-term project tracking or reporting to stakeholders who need to see "Scope vs. Progress."

4. How do you handle changing requirements in Agile project?

Agile "welcomes change," but it must be managed to avoid chaos:

- **Product Backlog:** All new requirements are sent to the Product Owner (PO), who adds them to the Product Backlog.
- **Prioritization:** The PO evaluates the new requirement against existing ones using techniques like **MoSCoW** (Must, Should, Could, Won't) or **WSJF** (Weighted Shortest Job First).
- **Sprint Integrity:** In Scrum, once a Sprint starts, the Sprint Goal should not be changed. New requirements are typically placed in the backlog for the *next* sprint.
- **Refinement:** The team discusses the new requirements during "Backlog Grooming" to ensure they are well-defined before being pulled into development.

5. How do you handle a situation, where testing is not completed within sprint?

If a ticket is not tested, it does not meet the **Definition of Done (DoD)**.

- **Mark as "Undone":** The ticket is moved back to the Product Backlog. It is **not** considered finished, and the team earns zero points for it in that sprint.
- **Re-estimation:** Before the next Sprint Planning, the PO and team re-evaluate the remaining work (testing) and re-estimate it.
- **Analyze in Retrospective:** The team must discuss *why* it wasn't finished. Was the story too big? Was there a "testing bottleneck" at the end of the sprint?

- **Solution:** To prevent recurrence, teams might adopt "**Shift Left**" testing (starting tests earlier) or ensure developers assist with testing tasks.

6. How do you estimate your tickets?

Agile teams use **Relative Sizing** rather than absolute hours to account for uncertainty:

- **Story Points:** A unit of measure expressing the overall effort, complexity, and risk of a task.
- **Planning Poker:** Team members use Fibonacci-sequenced cards (1, 2, 3, 5, 8, 13...) to vote on a ticket's size. If votes differ, the team discusses the reasons until they reach a consensus.
- **T-Shirt Sizing:** A high-level technique (XS, S, M, L, XL) often used for early-stage estimation of large epics.
- **Reference Stories:** Comparing a new ticket to a "Golden Story" (a previously completed task that everyone agrees is, for example, a "3") to determine its relative size.

7. What do you do when your automated suite is failed?

A failed automated suite (often in a CI/CD pipeline) requires immediate action to avoid "Red Build" fatigue:

1. **Investigate immediately:** Check the logs to see if it's a **real bug**, a **flaky test** (unstable test), or an **environment issue** (server down).
2. **Isolate the failure:** If it's a real bug, create a high-priority ticket. If it's a flaky test, quarantine it (skip/disable) so it doesn't block the rest of the team.
3. **Fix or Revert:** If a recent code merge caused the failure, either fix it immediately or revert the commit to restore the pipeline to a "Green" state.
4. **Root Cause Analysis (RCA):** If failures happen often, evaluate the locator strategies (e.g., using unstable IDs) or data management practices to improve suite reliability.